Reinforcement Learning for Stock Trading

1. Problem and Environment

This project models a simplified stock-trading problem as a finite-horizon, finite-state Markov decision process (MDP). The environment parameters are fixed as follows:

- Time horizon: $H = 20$
- Initial stock price: $S_0 = 100$
- Up factor: $u = 1.1$ (with probability $p = 0.5$)
- Down factor: $d = 0.9$ (with probability $1 - p = 0.5$)
- Discount factor: $\gamma = 1$

State Representation

The state is represented in code as a tuple:

```
s_t = (t, k, h)
```

where t is the current time step ($0 \le t \le H$), k is the number of upward price moves so far ($0 \le k \le t$), and $h \in \{0, 1\}$ is the holding status (0 = cash, 1 = stock). Since the stock price follows a binomial tree, the current price is fully determined by (t, k):

```
S_t = S₀ × u^k × d^(t−k)
```

so the price does not need to be stored separately in the state.

Actions and Validity

The action set is $A = \{\text{buy}, \text{hold}, \text{sell}\}$. Invalid actions are never allowed:

- If $h = 0$ (cash): valid actions are hold and buy
- If $h = 1$ (stock): valid actions are hold and sell

This constraint is enforced in both value iteration and Q-learning. In particular, random exploration samples uniformly from valid actions only.

Reward

The action is chosen at time t before the price moves to $t+1$. The reward is determined by the resulting holding position and the realized price movement:

- $r = \log(u) \approx 0.0953$ if holding stock and price moves up
- $r = \log(d) \approx -0.1054$ if holding stock and price moves down
- $r = 0$ if holding cash

Equivalently, the reward is the one-step log-return of the portfolio. Episodes terminate when $t = H$; no further reward is received.

2. Exact Solution by Value Iteration

Because this is a finite-horizon MDP, the exact benchmark is computed by backward dynamic programming. The terminal value is initialized as $V_H(s) = 0$ for all states. Then, for $t = H-1, H-2, \ldots, 0$, the Q-function is computed over valid actions only:

```
Q*(s, a) = Σ_{s'} P(s' | s, a) × [r(s, a, s') + γ × V*(s')]
V*(s)  = max_a Q*(s, a)
π*(s)  = argmax_a Q*(s, a)
```

When multiple actions tie, ties are broken in the fixed order: hold > buy > sell. This same tie-breaking rule is used in Q-learning policy extraction so that policy mismatch is measured consistently.

Optimal Value Function

An important observation is that V*(s) = 0 for every non-terminal state in this problem. This is not a bug but a direct mathematical consequence of the environment parameters.

The one-step expected reward of holding stock is:

```
E[r] = p·log(u) + (1−p)·log(d)
     = 0.5 × log(1.1) + 0.5 × log(0.9)
     = 0.5 × 0.0953 + 0.5 × (−0.1054) = −0.0025 < 0
```

This is negative because u × d = 1.1 × 0.9 = 0.99 < 1: the geometric mean of the stock price is slightly decreasing at every step. Holding stock therefore has a negative expected log-return per step, while holding cash always yields exactly 0. Since 0 > −0.0025, the optimal strategy is to never buy the stock at all, and the maximum achievable expected cumulative reward is 0. Backward dynamic programming confirms this: the optimal value function is V*(s) = 0 for every non-terminal state, and the optimal policy is to hold cash (H) in all states where h = 0. Figure 1 shows the optimal value function in a compact side-by-side layout. Since every optimal value equals 0, the figure emphasizes that the value function is uniform across cash and stock states.

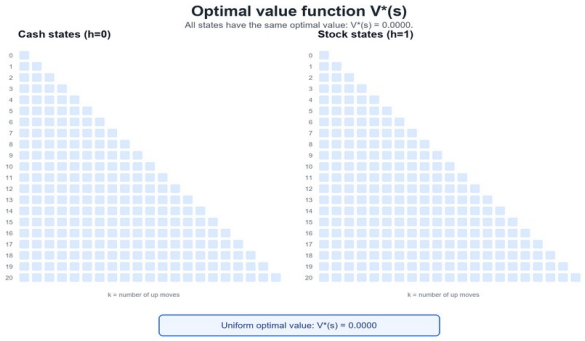


Figure 1: Optimal value function V*(s). Left panel: h=0 (cash). Right panel: h=1 (stock).

Optimal Policy

Figure 2 shows the optimal policy. Blue (B) = buy, green (H) = hold, red (S) = sell.

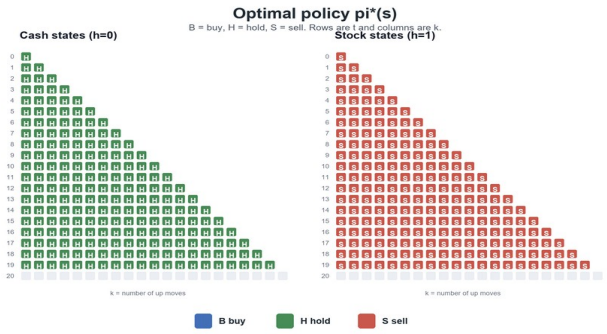


Figure 2: Optimal policy pi*(s). Left panel: h=0 (cash). Right panel: h=1 (stock).

3. Tabular Q-learning

The Q-learning implementation uses a tabular array Q(s, a) initialized to zero for all state-action pairs. Since the time index t is already embedded in the state tuple, no separate time-dependent table is required.

Update Rule

At each non-terminal state, the agent selects an action using ε-greedy exploration over valid actions only. The update rule is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \times [r + \max_{a'} Q(s', a') - Q(s, a)]$$

The maximization over a' uses only valid actions at the next state s'. If s' is terminal (t = H), the bootstrap term is zero.

Handling Valid/Invalid Actions

Both during exploration and policy extraction, the implementation strictly restricts action selection to the valid action set determined by the holding status h. Specifically:

- Exploration: random action is sampled uniformly from valid actions only
- Greedy action: argmax is taken over valid actions only
- Bootstrap target: max Q(s', a') is computed over valid actions at s'

4. Experimental Setup

The experiment tests 3 exploration schedules × 3 learning-rate schedules = 9 total combinations. Each combination is evaluated with 50 random seeds (0 through 49) and up to 5,000 episodes per seed. The same code supports 100,000 episodes via --episodes 100000.

Exploration Schedules (ε)

- **eps_linear_1_to_001:** linear decay from ε = 1.0 to ε = 0.01 over all episodes.
- **eps_exp_slow:** exponential decay from 1.0 to 0.01 with rate $\lambda = 3\times10^{-5}$.
- **eps_exp_fast:** exponential decay from 1.0 to 0.01 with rate $\lambda = 2\times10^{-4}$.

Learning-rate Schedules (α)

- **alpha_const_01:** constant α = 0.1.
- **alpha_sqrt:** $\alpha_n = 0.7 / \sqrt{(1 + n)}$, where n is the state-action visit count.
- **alpha_harmonic:** $\alpha_n = 1.0 / (10 + n)$.

Evaluation Metrics

For each final Q-table, three metrics are computed against the value-iteration benchmark:

- **Q-function error:** $\max_{s,a} |Q_{VI}(s,a) - Q_{QL}(s,a)|$
- **Value-function error:** $\max_s |V_{VI}(s) - V_{QL}(s)|$ where $V_{QL}(s) = \max_a Q_{QL}(s,a)$
- **Policy mismatch ratio:** fraction of non-terminal states where $\pi_{VI}(s) \neq \pi_{QL}(s)$

Mean and standard deviation are reported over the 50 seeds.

5. Results

Best Configuration

The best setting by mean Q-error was eps_linear_1_to_001__alpha_harmonic:

- Q-error mean / std: 0.040756 / 0.005026
- Value-error mean / std: 0.035183 / 0.005590
- Policy mismatch mean / std: 0.224619 / 0.018911

Full Summary Table

Table 1 shows all 9 settings sorted by mean Q-error (best to worst). Results are averaged over 50 seeds.

Setting	Q-err mean	Q-err std	V-err mean	V-err std	Mismatch mean	Mismatch std
eps_linear + alpha_harmonic	0.0408	0.0050	0.0352	0.0056	0.2246	0.0189
eps_exp_fast + alpha_harmonic	0.0415	0.0043	0.0361	0.0046	0.2594	0.0196
eps_exp_slow + alpha_harmonic	0.0432	0.0061	0.0379	0.0063	0.3290	0.0241
eps_linear + alpha_const_0.1	0.0745	0.0124	0.0693	0.0123	0.1727	0.0145
eps_exp_fast + alpha_const_0.1	0.0961	0.0132	0.0912	0.0130	0.2403	0.0173
eps_linear + alpha_sqrt	0.1276	0.0169	0.1228	0.0166	0.2061	0.0163
eps_exp_fast + alpha_sqrt	0.1331	0.0140	0.1282	0.0139	0.2618	0.0181
eps_exp_slow + alpha_const_0.1	0.1479	0.0136	0.1430	0.0136	0.3925	0.0244
eps_exp_slow + alpha_sqrt	0.1492	0.0109	0.1441	0.0109	0.3872	0.0259

Table 1: Experiment summary — all 9 hyperparameter combinations, sorted by Q-error mean.

Learning Curves

Figure 3 shows the Q-error versus number of training episodes for the top 5 settings using a compact log-scale episode axis. The harmonic learning-rate variants converge to much lower final errors than the constant learning-rate variants.

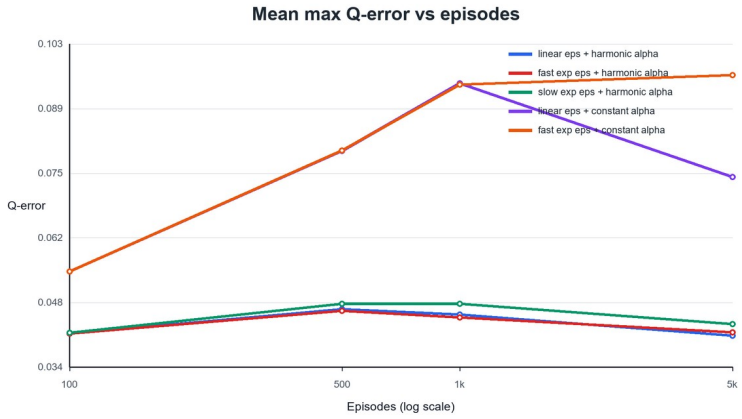


Figure 3: Mean max Q-error vs. episodes for the top 5 configurations (50 seeds).

Policy Comparison

Figure 4 compares the optimal policy from value iteration with the policy extracted from the best Q-learning run. Thick black borders mark states where the two policies disagree.

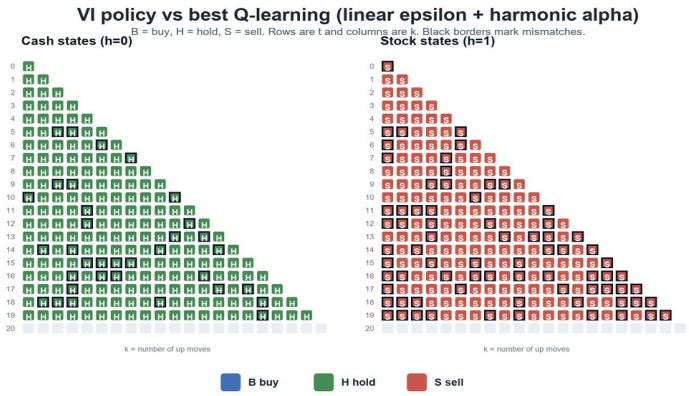


Figure 4: VI policy with thick borders marking mismatches with best Q-learning (eps_linear + alpha_harmonic).

6. Analysis

Why Harmonic Learning Rate Works Best

The harmonic learning-rate schedule (alpha_n = 1/(10+n)) gave the smallest Q-function approximation errors across all three exploration schedules. This schedule updates aggressively early in training when visit counts n are small, but becomes increasingly conservative as more samples are collected. In a finite tabular MDP where the exact optimal Q-values are stable targets, this reduction in step size helps stabilize estimates once they are close to the dynamic-programming benchmark. The classical Robbins-Monro conditions for Q-learning convergence require $\Sigma\alpha = \infty$ and $\Sigma\alpha^2 < \infty$; the harmonic schedule satisfies both.

Constant vs. Sqrt Learning Rate

The constant learning rate (alpha = 0.1) sometimes produced a lower policy mismatch ratio than the harmonic schedule, particularly with linear epsilon decay. However, its Q-error and value-error were significantly larger. This means the greedy action was often correct, but the Q-values themselves remained numerically inaccurate. Since the project goal is approximation of the value-iteration solution (not just behavioral matching), Q-error and value-error are the more informative metrics, and the harmonic schedule dominates on both.

The square-root schedule (alpha_n = 0.7/sqrt(1+n)) performed worse than harmonic in these experiments. Its step size decays more slowly, so Q-estimates continue to shift substantially even after many visits. In a small finite MDP where optimal Q-values are well-defined constants, this slower decay leads to noisier final estimates.

Effect of Epsilon Schedule

The slow exponential exploration schedule performed poorly regardless of learning rate. With only 5,000 episodes and a low decay rate ($\lambda = 3\times10^{-5}$), epsilon remains near 1.0 for the majority of training, so the agent continues taking many random actions late in the learning phase. This slows convergence of both the Q-values and the greedy policy.

The fast exponential schedule ($\lambda = 2\times10^{-4}$) performed better than the slow one, but still slightly worse than linear decay when paired with harmonic learning rates. Linear decay provides a more predictable trade-off between exploration and exploitation, which appears to be beneficial in this structured finite MDP.

Overall Conclusion

The best-performing configuration was linear epsilon decay combined with harmonic state-action learning rates (eps_linear_1_to_001__alpha_harmonic). It achieved a mean Q-error of 0.0408 and a mean value-function error of 0.0352 over 50 seeds. This combination balanced sufficient early exploration with effective exploitation later, and reduced the learning rate enough for the final Q-table to closely approximate the value-iteration benchmark.